Prometheus
CORPORATION

TECHNICAL DESIGN BRIEF

Football Match Outcome Probability Engine

System Design & Technical Architecture Brief

Project	Football Match Outcome Probability Engine
Version	1.0
Date	March 2026
Prepared by	Nzioki Dennis — Prometheus Corporation
Status	Draft — For Client Review

1. Executive Summary

This document defines the complete technical design for a Football Match Outcome Probability Engine. The system accepts bookmaker odds for any football fixture, converts those odds into probability weights, and runs hundreds of simulated match iterations to produce a statistically stable prediction across three possible outcomes: Home Win, Draw, and Away Win.

The engine is designed to be fast, reusable for any fixture, mathematically sound, and extensible toward future enhancements such as team-performance weighting and live odds integration.

2. Problem Statement

Bookmaker odds reflect the market's implied probability for each match outcome. However, raw odds include bookmaker margin (overround) and do not directly express clean probabilities. A single calculation is also statistically unstable. The system must:

- Extract clean implied probabilities from odds.
- Remove bookmaker margin through normalisation.
- Simulate many outcomes to produce a stable distribution.
- Be reusable across any fixture without code changes.

3. System Architecture

The engine is structured as a linear processing pipeline. Each stage has a single responsibility and passes its output to the next stage.

Stage	Module	Responsibility
1	Input Layer	Accept odds values and simulation count from user
2	Validation Module	Reject invalid, missing, or malformed input
3	Odds Converter	Transform odds into normalised probability weights
4	Simulation Engine	Run Monte Carlo iterations using probability weights
5	Aggregator	Count frequency of each outcome across all simulations
6	Output Module	Display percentage probability for each outcome

4. Mathematical Model

4.1 Odds to Probability Conversion

Bookmaker odds are the inverse of probability. For odds O, the implied probability P is:

```
P = 1 / O
```

Because bookmaker overround causes raw probabilities to sum to more than 1.0, each value is normalised by dividing by the total:

```
P_normalised = P_i / sum(P_all)
```

Outcome	Odds	Raw P	Normalised P
Home Win (A)	2.10	0.476	45.2%
Draw (X)	3.20	0.312	29.6%
Away Win (B)	3.60	0.278	26.4%
Total	—	1.066	~100%

Table 1 — Example odds normalisation (Arsenal vs Chelsea)

4.2 Monte Carlo Simulation

Monte Carlo simulation runs N independent trials. In each trial a random outcome is selected using the normalised probability weights. Outcome frequency across all trials converts to the final probability estimate. More trials produce a more stable result.

- Minimum recommended iterations: 500
- Standard run: 1 000
- High-precision run: 10 000+
- Maximum cap (performance): 100 000

Outcome	Simulated Count (n=500)	Probability
Home Win	238	47.6%
Draw	152	30.4%
Away Win	110	22.0%

Table 2 — Example simulation output

5. Functional Requirements

ID	Requirement	Detail
FR-01	Match Input	Accept team names, three odds values, simulation count
FR-02	Odds Conversion	Convert raw odds to normalised probability weights
FR-03	Simulation Engine	Run weighted random selection N times (min 500)
FR-04	Result Output	Display Win / Draw / Loss percentages
FR-05	Reusability	Accept new odds and recalculate without restart
FR-06	Input Validation	Reject negative, zero, non-numeric, or missing odds
FR-07	Seed Support	Optional fixed random seed for reproducible results

6. Non-Functional Requirements

Property	Target
Speed	10 000 simulations must complete in under 1 second
Reusability	Any fixture processable without code modification
Accuracy	Probability error < 1% at 1 000+ iterations
Scalability	Simulation count configurable at runtime
Reliability	Graceful error handling on invalid input
Determinism	Fixed seed produces identical output across runs

7. Edge Cases & Risk Handling

Case	Risk	Mitigation
Negative or zero odds	Division by zero / invalid P	Reject; show error
Non-numeric input	Runtime crash	Type validation on entry
Simulations < 100	Unstable probabilities	Enforce minimum = 100
Simulations > 100 000	Performance lag	Cap at 100 000
Equal odds (e.g. 2.5 all)	Each outcome ~33.3%	No special handling needed
Extreme odds (e.g. 1000+)	Near-zero probability	Float precision; display 0.1%+
Probability sum != 1.0	Distribution error	Assert tolerance ±0.000001

8. Security Considerations

Although Version 1 is a standalone local tool, the following controls are applied as baseline practice in anticipation of future web deployment:

Threat	Control
Input injection	Strict numeric-only validation on all odds fields
API abuse	Rate limiting on any future web endpoint
Denial of service	Hard cap on maximum simulation iterations
Data manipulation	Server-side re-validation if odds arrive via API

9. Technology Stack

Phase 1 — Core Engine

Component	Technology	Rationale
Primary language	Python 3.x	Strong numerical libraries, rapid development
Simulation	random / numpy	Fast weighted random selection
Data handling	pandas	Optional; useful for historical data in Phase 2
Interface (v1)	CLI	Minimal setup; fastest delivery
Interface (v2)	Web (Flask)	Browser-accessible; shareable

10. Deliverables

Phase 1 (MVP)

- Probability conversion module
- Monte Carlo simulation engine
- Command-line interface
- Input validation layer
- Formatted probability output

Phase 2 (Enhancement)

- Historical match performance weighting
- Poisson distribution goal model
- Elo rating integration
- Web interface
- Automated odds ingestion via API

11. Example Workflow

The following illustrates a complete end-to-end system execution:

Step	Detail
Input	Arsenal vs Chelsea | Odds: A=2.10, X=3.20, B=3.60 | Simulations: 1 000
Conversion	Raw probabilities: A=0.476, X=0.312, B=0.278 (sum=1.066)
Normalise	A=44.7%, X=29.3%, B=26.1%
Simulate	1 000 iterations run using normalised weights
Output	Arsenal Win: 46.2% | Draw: 29.8% | Chelsea Win: 24.0%

12. Future Roadmap & Extensions

Enhancement	Description
Poisson Goal Model	Predict scorelines in addition to match outcomes
Elo Rating Adjustment	Weight probabilities by relative team strength
Bayesian Updating	Incorporate recent form to refine base odds
Live Odds API	Ingest real-time bookmaker odds automatically
Machine Learning Layer	Train on historical data to improve prediction accuracy
Web Dashboard	Browser UI for non-technical users